PEtab problems in dMod2

Simon Beyer

2026-10-18

Contents

1 Overview **2**

2 PEtab objects and their dMod2 counterparts **2**

3 Importing a problem **2**
3.1 Prerequisites . . . 3

4 Exporting a problem **3**

5 A worked example: Boehm **4**
5.1 Import and evaluate . . . 4
5.2 Predict and plot . . . 4
5.3 Write it back out . . . 5

6 How the importer works **6**

7 Priors **6**

8 How the exporter works **7**

9 Feature coverage **8**
9.1 SBML model . . . 8
9.2 PEtab tables . . . 8
9.3 Noise distributions . . . 9
9.4 Export caveats . . . 9

References **10**

1 Overview

PEtab [1] is a file format for parameter estimation problems in systems biology: a YAML manifest, an SBML model [2] and a set of TSV tables. dMod2 imports such a problem as a fully assembled fit problem in one call and exports dMod problems in the same format.

PEtab refers to v2 unless stated otherwise. Version 1 problems are read as well and normalised onto the same internal shape.

2 PEtab objects and their dMod2 counterparts

dMod2 composes a fit problem from function objects, `g * x * p`, closed by an objective [3]. The importer fills these slots:

PEtab	dMod2
SBML reactions, rules, events	`reactions` (an `eqnlist`), `odemodel`, `x = Xs(odemodel)`
`observables.tsv` formulas	`g = Y(observables)`
`observables.tsv` noise formulas	`e`, the error model, or `NULL` when every noise formula is constant
`parameters.tsv`, `conditions.tsv`	`p = P(trafo)`, one branch per condition
`measurements.tsv`	`dataList`, a `datalist` keyed by condition
`experiments.tsv`	simulation start times and mid-run events
`parameters.tsv` priors	a prior term added to the objective

PEtab declares a scale per parameter (`lin`, `log`, `log10`). dMod2 wraps `p` so that the optimiser sees each parameter on its scale and the chain rule applies through the composition; `bestfit` is therefore on the PEtab scale, not the linear one.

3 Importing a problem

```
petab <- importPEtab("problem.yaml", backend = "cppDE")
```

`importPEtab()` parses the manifest, reads the SBML through libsbml, generates and compiles the right-hand side with its sensitivity equations, and returns an object of class `petabproblem`.

argument	meaning
`yamlPath`	path to the YAML manifest
`backend`	`"cppDE"` (default, handles events and second-order sensitivities) or `"deSolve"`
`compile`	`FALSE` reuses an already compiled model of the same name
`cores`	conditions evaluated in parallel inside one objective call
`modelname`	base name for the generated sources
`outdir`	where the generated sources and the shared object are written
`options`, `optionsSens`	solver settings, forwarded to `Xs()`

`outdir` defaults to the working directory. A problem with many conditions writes one source per condition, so a scratch folder is preferable. Solver settings default to the backend's; reproducing reference values computed at tighter tolerances requires passing those tolerances.

The returned object is a plain list whose slots are ordinary dMod objects:

slot	what it is
`dataList`	the measurements as a `datalist`
`reactions`	the reaction network as an `eqnlist`
`odemodel`, `x`	compiled model and prediction function
`g`, `e`	observation function and error model
`p`	parameter transformation, one branch per condition
`prd`	the composite `g * x * p`
`obj`	`normL2(dataList, prd, errmodel = e)`, fixed parameters baked in
`bestfit`	`nominalValue` per estimated parameter, on the PEtab scale
`parlower`, `parupper`	bounds, likewise on the PEtab scale

With the fixed parameters inside `obj`, the likelihood at the published optimum is

```
petab$obj(petab$bestfit)$value
```

and a fit is the usual dMod call, with the bounds passed to the optimiser, not to the objective:

```
fit <- trust(petab$obj, petab$bestfit, rinit = 1, rmax = 10,
             parlower = petab$parlower, parupper = petab$parupper)
```

3.1 Prerequisites

Reading the SBML model requires `python-libsbml`, which dMod2 provisions on first use in a Python environment managed by `reticulate`. Setting `DMOD_LIBSBML_PYTHON` to an existing interpreter uses that one instead. The YAML and TSV readers, `readPEtabYaml()` and `readPEtabTables()`, work without libsbml.

4 Exporting a problem

Imported problems. `exportPEtabObject()` writes the manifest, the SBML and every table. The tables it read are reproduced, including prior declarations, per-row noise parameters and the period schedule.

```
exportPEtabObject(petab, dir = "out", formatVersion = "2.0.0")
```

argument	meaning
`petab`	an imported problem, or a list with the same slots
`dir`	output directory, created if missing
`modelID`	model id in the manifest, defaults to the imported one
`formatVersion`	`"2.0.0"` or `"1"`
`overwrite`	overwrite an existing directory

Problems built by hand. `exportPEtab()` takes the dMod objects directly rather than a `petabproblem`:

```
exportPEtab(data = myData, reactions = myEqnlist,
            observables = myObservables,
            p = myTrafo, pouter = myPars,
            errors = myErrorModel,
            lower = myLower, upper = myUpper,
            parameterScale = "log10",
            dir = "out", formatVersion = "2.0.0")
```

`conditions.tsv` is recovered from the per-condition structure of the trafo `p`. A symbol that `p` references but that is neither estimated nor declared fixed raises an error.

5 A worked example: Boehm

The package ships the JAK/STAT5 problem of Boehm et al. [4], a PEtab benchmark problem, so the example runs as written.

```
petab_dir <- system.file("extdata/petab_boehm", package = "dMod2")
list.files(petab_dir)
#> [1] "Boehm.yaml"
#> [2] "experimentalCondition_Boehm_JProteomeRes2014.tsv"
#> [3] "measurementData_Boehm_JProteomeRes2014.tsv"
#> [4] "model_Boehm_JProteomeRes2014.xml"
#> [5] "observables_Boehm_JProteomeRes2014.tsv"
#> [6] "parameters_Boehm_JProteomeRes2014.tsv"
```

The manifest and tables can be inspected before compilation:

```
yamlPath <- file.path(petab_dir, "Boehm.yaml")
tables   <- readPEtabTables(yamlPath)

obs <- tables$observables
data.frame(id = obs$observableId, noise = obs$noiseFormula)
#>            id                        noise
#> 1 pSTAT5A_rel noiseParameter1_pSTAT5A_rel
#> 2 pSTAT5B_rel noiseParameter1_pSTAT5B_rel
#> 3 rSTAT5A_rel noiseParameter1_rSTAT5A_rel
```

The observable formulas:

```
writeLines(strwrap(paste0(obs$observableId, " = ",
                          obs$observableFormula),
                   width = 68, exdent = 4))
#> pSTAT5A_rel = (100 * pApB + 200 * pApA * specC17) / (pApB + STAT5A
#>     * specC17 + 2 * pApA * specC17)
#> pSTAT5B_rel = -(100 * pApB - 200 * pBpB * (specC17 - 1)) / ((STAT5B
#>     * (specC17 - 1) - pApB) + 2 * pBpB * (specC17 - 1))
#> rSTAT5A_rel = (100 * pApB + 100 * STAT5A * specC17 + 200 * pApA *
#>     specC17) / (2 * pApB + STAT5A * specC17 + 2 * pApA * specC17 -
#>     STAT5B * (specC17 - 1) - 2 * pBpB * (specC17 - 1))
```

The model has eight STAT5 species, six estimated rate constants, three estimated noise standard deviations and two fixed scaling parameters; all estimated parameters are on `log10` scale.

5.1 Import and evaluate

```
wd <- tempfile("petab_vignette_"); dir.create(wd)
petab <- importPEtab(yamlPath, backend = "cppDE",
                     modelname = "boehm_vig", outdir = wd)
```

The objective, $-2\log L$, at the published maximum-likelihood estimate:

```
petab$obj(petab$bestfit)$value
#> [1] 276.4441
```

5.2 Predict and plot

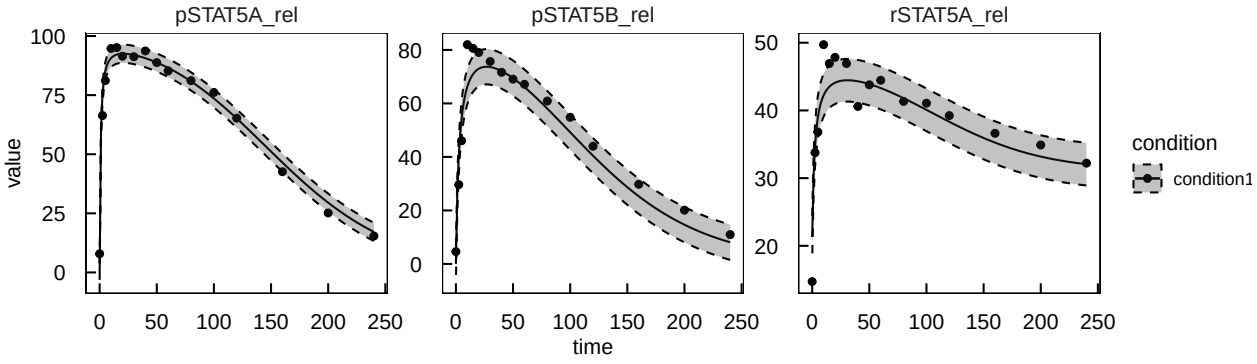
`prd` returns one trace per observable, and `plot()` draws them against the data. The band spans one standard deviation either side, taken from the error model: `as.data.frame(errfn = )` evaluates it alongside the prediction and returns `sigma` per row.

```r
library(ggplot2)

fixed <- attr(petab, "petab_meta")$fixed
times <- seq(0, 240, length.out = 200)
pred  <- petab$prd(times, c(petab$bestfit, fixed), deriv = FALSE)

prdout <- as.data.frame(pred, errfn = petab$e)

plot(pred, petab$dataList) +
  geom_ribbon(data = prdout,
              aes(x = time, ymin = value - sigma, ymax = value + sigma),
              linetype = "dashed", alpha = 0.3)
```

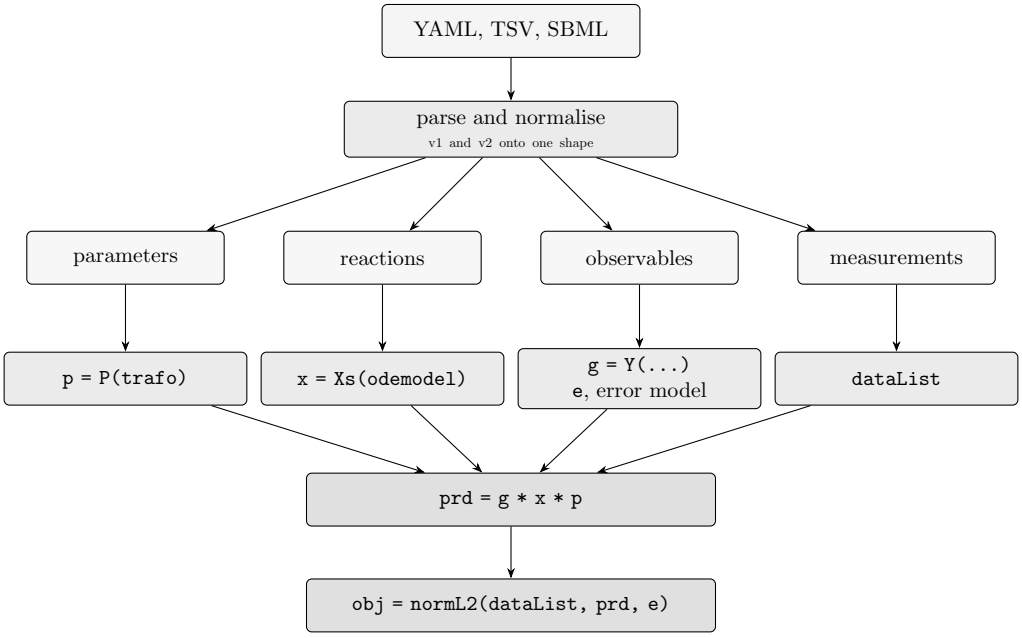


5.3 Write it back out

Boehm is a v1 problem, so exporting it as v2 linearises its `parameterScale`, with a warning.

```r
exportDir <- file.path(wd, "export"); dir.create(exportDir)
exported <- exportPEtabObject(petab, dir = exportDir,
                              formatVersion = "2.0.0",
                              overwrite = TRUE)
list.files(exportDir)
#> [1] "boehm_vig.xml"               "boehm_vig.yaml"
#> [3] "conditions_boehm_vig.tsv"    "experiments_boehm_vig.tsv"
#> [5] "measurements_boehm_vig.tsv" "observables_boehm_vig.tsv"
#> [7] "parameters_boehm_vig.tsv"
```

6 How the importer works



Extensive against intensive flux. SBML kinetic laws are in amount per time, dMod2 rates in concentration per time. On import, each kinetic law is divided by the volume of its home compartment, $\text{rate}_{\text{dMod}} = K_{\text{SBML}}/V_{\text{home}}$, and on export multiplied by it again. The home compartment is the shared compartment of the educts, or of the products for a pure synthesis. Assignment rules are inlined into rates and initial expressions before the `eqnlist` is built.

Experiments as period sequences. An experiment is an ordered list of `(time, conditionId)` pairs. A leading `-inf` period is the preequilibration: the steady state of the autonomous system under that condition, computed by `Pimpl`. Events do not act during it. The first finite period sets the simulation start, passed to the objective as the first time of the condition's grid, `normL2(times = list(<condition> = <start>))`, so initial values apply there rather than at zero. Each later period becomes an event that applies its condition overrides at its start time. The number of periods is unlimited.

Event targets that are not states. Solvers assign only to states, but PEtab and SBML events may also assign to a parameter or a compartment volume. Such a target is promoted to a state with rate zero and its SBML default as initial value; export reverses the promotion.

Placeholders per observable. `measurements.tsv` may override `observableParameter${k}_${id}` and `noiseParameter${k}_${id}` per row. The observable id suffix keeps substitutions from colliding across observables. The rows of one condition merge into one sub-condition whose trafo applies one substitution per observable; a further sub-condition is created only when an observable appears with conflicting values. Numeric noise is folded into the data column and leaves the error model `NULL`, `normL2`'s fast path; symbolic noise becomes an inner parameter of the trafo.

Steady-state measurements. A measurement at `time = inf` is evaluated at time `1e10` and exported as `inf`.

7 Priors

A parameter row may declare a `priorDistribution` with `priorParameters`. With π_i the prior density of parameter i, truncated to its bounds, `obj()` returns the $-2\log$ posterior

$$-2\log L - 2\sum_i \log \pi_i$$

rather than $-2\log L$.

Separating prior and likelihood. The prior term $-2\sum_i \log \pi_i$ is returned as the attribute `prior` of the result; subtracting it leaves $-2\log L$, that is -2 times the `llh` PEtab publishes:

```
res <- petab$obj(petab$bestfit)
prior <- attr(res, "prior")

c(total      = unname(res$value),
  prior      = if (is.null(prior)) 0 else unname(prior),
  likelihood = unname(res$value) - if (is.null(prior)) 0 else unname(prior))
#>      total      prior likelihood
#>   276.4441     0.0000   276.4441
```

Boehm declares no prior, so the attribute is absent and `total` equals `likelihood`.

Bounds are not a prior. `lowerBound` and `upperBound` reach the fit as `parlower` and `parupper` and never enter the objective, so a parameter without a declared distribution contributes nothing. Implementations that read bounds as an implicit uniform prior report a posterior that includes a constant dMod2 leaves out.

Truncation. PEtab truncates every prior to the parameter bounds and renormalises it to the mass between them; dMod2 does the same. The renormalisation is constant in the parameter: it shifts the objective value but not gradient or Hessian, so the fit is unaffected.

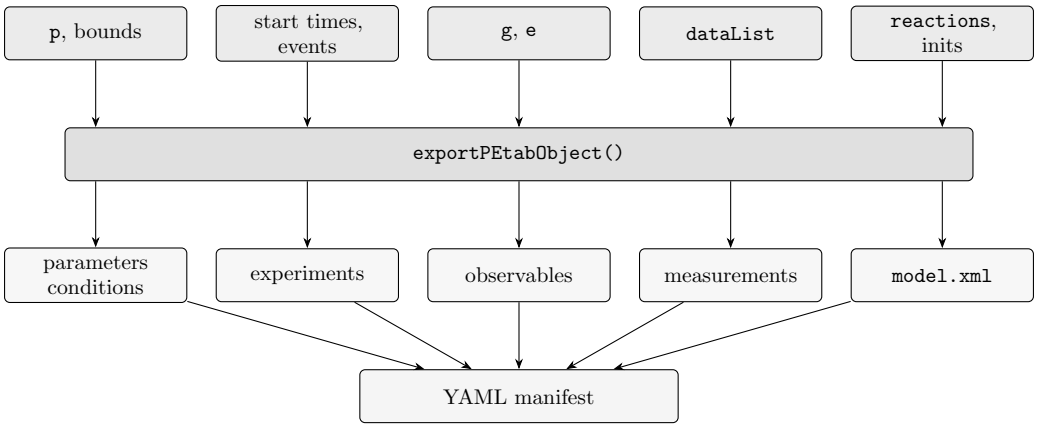
Families and their parameterisation.

`priorDistribution`	parameters	note
`uniform`	a, b	density $1/(b-a)$ on $[a, b]$
`normal`	mean, sd	
`laplace`	mu, b	
`cauchy`	x0, gamma	
`chisquare`	df	
`exponential`	**scale**	not a rate
`gamma`	shape, **scale**	not a rate
`rayleigh`	sigma	
`log-normal`	meanlog, sdlog	density in the parameter, so it includes the change of variables
`log-laplace`	mu, b	likewise
`log-uniform`	a, b	likewise

Each name also exists with a `parameterScale` prefix. Prefixed priors act on the parameter as the optimiser sees it, after the scale wrap; plain ones act on the linear value. A plain `normal` or `laplace` on a non-`lin` `parameterScale` is rejected: the two readings differ, and choosing one silently would move the optimum.

8 How the exporter works

The exporter reads the same slots and writes one table each. Besides undoing the compartment scaling and the promotion of event targets, it reconstructs the period schedule from the simulation start times and the mid-run events.



`exportPEtab()` takes the same pieces as separate arguments.

9 Feature coverage

30 of the 31 v2 cases of the official PEtab test suite reproduce the published likelihood and pass an export-to-v2 and reimport round trip. Case 0023 fires an event during preequilibration, which dMod2 does not do.

Of the 35 problems in the Benchmark-Models collection, 33 import, evaluate their objective with derivatives, export to v2 and reimport. Left out are `Froehlich_CellSystems2018`, whose 9570 conditions each produce their own generated transformation, and `Laske_PLOSComputBiol2019`, where one observable divides by a sum of species that is zero at the initial time.

9.1 SBML model

feature	dMod2	notes
reactions and kinetic laws	yes	divided by the home compartment volume
compartments, multiple compartments	yes	first class on `eqnlist`
species in amounts or concentrations	yes	`hasOnlySubstanceUnits` respected
`<initialAssignment>`	yes	on species, parameters and compartments
`<assignmentRule>`	yes	inlined before the `eqnlist` is built
`<rateRule>`	yes	on species, and on parameters, which become states
`<event>`, time trigger	yes	rewritten as a dMod event
`<event>`, state trigger	yes	as a root condition
event assigning to a compartment	yes	contained species rescale, amounts conserved
`<functionDefinition>`	yes	inlined by libsbml
`piecewise` in a formula	yes	grouped into a symbolic piecewise
`<algebraicRule>`	no	dMod2 has no DAE solver
model languages other than SBML	no	only SBML is read

9.2 PEtab tables

feature	dMod2	notes
`parameterScale`	yes	`lin`, `log`, `log10`, folded into the trafo
`observableTransformation`	yes	`lin`, `log`, `log10` per observable
`conditions.tsv`, long format	yes	pivoted to wide for the trafo builder
condition target on a parameter or compartment	yes	promoted to a state when an event assigns to it
per-row `observableParameters`, `noiseParameters`	yes	numeric and symbolic
noise formula naming its own observable	yes	resolved from the observable equations
`experiments.tsv`, any number of periods	yes	preequilibration, start time, mid-run events
event during preequilibration	no	preequilibration is a steady state
measurement at `time = inf`	yes	evaluated at the equilibration horizon
`mapping.tsv`	yes	textual rewrite of entity ids
several models in one manifest	yes	composed with `+`, condition keys prefixed
priors, all eleven families	yes	truncated to the bounds and renormalised
model without species	yes	a parameter-only problem
`visualisation.tsv`	no	ignored

9.3 Noise distributions

The supported noise distributions follow from dMod2's objective, `normL2`.

`noiseDistribution`	dMod2	notes
`normal`	yes	direct
`log-normal`	yes	through `observableTransformation = log`
`laplace`, `log-laplace`	no	rejected at import; an L1 residual is not implemented

9.4 Export caveats

caveat	notes
`parameterScale` on a v2 export	linearised on disk with one warning; v2 has no such column
`log10` observable transformation on a v2 export	rewritten as `log-normal` noise; v2 has no `log10`
`<algebraicRule>` in the source model	absent from the dMod model and therefore from any SBML written back
experiment identity	experiments are keyed by their starting condition, so the exported `experimentId` need not match the imported one
initial values set per condition in the trafo only	not written to a v2 condition table; a v1 export keeps them
parameter scales of a hand-built problem	read from `attr(bestfit, "petab_scales")`, `lin` without it

References

[1] L. Schmiester *et al.*, "PEtab: Interoperable specification of parameter estimation problems in systems biology," *PLOS Computational Biology*, vol. 17, no. 1, p. e1008646, 2021, doi: 10.1371/journal.pcbi.1008646.

[2] M. Hucka *et al.*, "The systems biology markup language (SBML): A medium for representation and exchange of biochemical network models," *Bioinformatics*, vol. 19, no. 4, pp. 524–531, 2003, doi: 10.1093/bioinformatics/btg015.

[3] D. Kaschek, W. Mader, M. Fehling-Kaschek, M. Rosenblatt, and J. Timmer, "dMod: Efficient parameter estimation and uncertainty analysis for dynamic models," *Journal of Statistical Software*, vol. 88, no. 10, pp. 1–32, 2019, doi: 10.18637/jss.v088.i10.

[4] M. E. Boehm, L. Adlung, M. Schilling, S. Roth, U. Klingmüller, and W. D. Lehmann, "Identification of isoform-specific dynamics in phosphorylation-dependent STAT5 dimerization by quantitative mass spectrometry and mathematical modeling," *Journal of Proteome Research*, vol. 13, no. 12, pp. 5685–5694, 2014, doi: 10.1021/pr5006923.